AI Driven Full Stack Development (AI308B)

Moodle MSE1 Submission Format

Name: Priyesh Kumar
Branch: [Enter Branch]
Roll Number: [Enter Roll Number]
Section: [Enter Section]
Shift: [Enter Shift]
Case Study Name: Hostel Management System

---

1. GitHub Repository Link

https://github.com/Priyesh288/mse-1-aifsd-priyesh-kumar

---

2. Render Deployment Links for all Routes

(Wait for final Render deployment. Replace below links when deployed)

- **Frontend URL:** `[Render Frontend URL]`
- **Backend API URL:** `[Render Backend URL]/api/students`

---

3. Project code which is used in the project development.

- **`index.js`**

```
require('dotenv').config();
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
const Student = require('./model');

const app = express();

// Middleware
app.use(cors());
app.use(express.json());

// In-Memory Fallback if MongoDB is not running locally
let useMongoDB = false;
let fallbackDb = [
    { _id: '1', name: 'Rahul Sharma', roomNo: '101-A', course: 'B.Tech CSE', status:
'Active', createdAt: new Date() },
    { _id: '2', name: 'Aman Verma', roomNo: '205-B', course: 'B.Tech ECE', status:
'Active', createdAt: new Date() }
];

// Database Connection
mongoose.connect(process.env.MONGODB_URI, {
    serverSelectionTimeoutMS: 3000 // 3 seconds timeout
```

```
})
    .then(() => {
        console.log('✅ Connected to MongoDB database (hostelsys)');
        useMongoDB = true;
    })
    .catch((err) => {
        console.log('⚠️ MongoDB not detected running locally. Using in-memory array for
demo purposes so it works instantly.');
    });

// Get all students (Read)
app.get('/api/students', async (req, res) => {
    try {
        if (useMongoDB) {
            const students = await Student.find().sort({ createdAt: -1 });
            return res.status(200).json(students);
        } else {
            return res.status(200).json(fallbackDb.sort((a,b) => b.createdAt -
a.createdAt));
        }
    } catch (err) {
        res.status(500).json({ error: 'Failed to fetch students', details: err.message });
    }
});

// Add a new student (Create)
app.post('/api/students', async (req, res) => {
    try {
        const { name, roomNo, course, status } = req.body;
        if (!name || !roomNo || !course) return res.status(400).json({ error: 'Name, Room
No, and Course are required fields.' });

        if (useMongoDB) {
            const newStudent = new Student({ name, roomNo, course, status: status ||
'Active' });
            const savedStudent = await newStudent.save();
            return res.status(201).json(savedStudent);
        } else {
            const newStudent = { _id: Date.now().toString(), name, roomNo, course, status:
status || 'Active', createdAt: new Date() };
            fallbackDb.push(newStudent);
            return res.status(201).json(newStudent);
        }
    } catch (err) {
        res.status(500).json({ error: 'Failed to add student', details: err.message });
    }
});

// Delete a student (Delete)
app.delete('/api/students/:id', async (req, res) => {
    try {
        const studentId = req.params.id;
```

```
        if (useMongoDB) {
            const deletedStudent = await Student.findByIdAndDelete(studentId);
            if (!deletedStudent) return res.status(404).json({ error: 'Student not found.'
});
        } else {
            const initialLength = fallbackDb.length;
            fallbackDb = fallbackDb.filter(s => s._id !== studentId);
            if (fallbackDb.length === initialLength) return res.status(404).json({ error:
'Student not found.' });
        }
        res.status(200).json({ message: 'Student deleted successfully.', id: studentId });
    } catch (err) {
        res.status(500).json({ error: 'Failed to delete student', details: err.message });
    }
});

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`🚀 API Server running on http://localhost:${PORT}`));
```

- `.git`

(The `.git` folder contains version control history. Below is the list of internal Git structure files generated automatically by Git init.)

```
.git/
├── HEAD
├── config
├── description
├── hooks/
├── info/
├── objects/
└── refs/
```

- `.env`

```
PORT=5000
MONGODB_URI=mongodb://localhost:27017/hostelsys
```

- `model.js` **(Extra added for clarity)**

```
const mongoose = require('mongoose');

const studentSchema = new mongoose.Schema({
    name: { type: String, required: true, trim: true },
    roomNo: { type: String, required: true, trim: true },
    course: { type: String, required: true, trim: true },
    status: { type: String, enum: ['Active', 'Pending'], default: 'Active' }
}, { timestamps: true });
```

```
const Student = mongoose.model('Student', studentSchema);
module.exports = Student;
```

---

4. Screenshots of All APIs testing after deployment on Render.

(Leave empty until Render deployment is fully completed. To generate these, use Postman/Browser to hit your Render URLs and attach screenshots here)

[Insert Screenshots Here]

---

5. Screenshot of VS Code Project Structure.

(Open VS Code, expand the `mse 1 aifsd priyesh kumar` folder on the left side, take a screenshot and paste it below)

[Insert VS Code Screenshot Here]